A 4.99M-Parameter SE-ResNet for CIFAR-10: Design, Training Recipe, and Generalization to a Shifted Test Set

Team KHL — Deep Learning Project 1, Fall 2026
Shreyas Kaldate · Adhyayan Verma · Sumit Shah
Code: https://github.com/Shreyas191/cifar10-resnet-5m

1 Overview and summary of findings

We designed and trained from scratch a residual network with **4,988,274 trainable parameters** (budget: 5,000,000) for the CIFAR-10 classification competition. Our final model, a three-stage ResNet with Squeeze-and-Excitation (SE) attention and residual blocks arranged $N = (3, 4, 3)$ over channels $C = (64, 128, 256)$, reaches **97.00% top-1 accuracy on the 10,000-image CIFAR-10 test split** (97.20% with horizontal-flip test-time augmentation) after 200 epochs on a single T4 GPU (2.0 h). On the competition's held-out set it scored **0.891 on the public leaderboard and 0.869 on the private leaderboard**, moving up eight places between the two (rank 41 private).

Our main findings:

1. **Under a 5M budget, depth in the 64/128/256-channel stages beats a fourth 512-channel stage.** A standard ResNet-18 spends 8.4M of its 11.2M parameters in its last stage, which operates on 4×4 feature maps. Dropping that stage and reinvesting the budget in extra blocks at higher resolution, plus almost-free SE attention (37k parameters), gave a better accuracy-per-parameter trade-off.
2. **The training recipe matters as much as the architecture.** Heavy regularization (TrivialAugment, random erasing, CutMix/MixUp, label smoothing), a long cosine schedule, and an exponential moving average (EMA) of weights took the same architecture from the low-90s to 97%. The EMA model was consistently 1–5 points above the raw weights during training and removed the need to pick a "lucky" epoch.
3. **The competition test set is not drawn from CIFAR-10's test split.** Our analysis of the 2,000 test images (Section 2.5) shows clean natural images, ∼200 per class, with slightly different statistics—the signature of CIFAR-10.1 [11], a re-collection of CIFAR-10 from the same source designed to measure overfitting to the benchmark. Every team's score dropped by 6–10 points relative to CIFAR-10 accuracy. Because out-of-distribution accuracy on such sets is a near-linear function of in-distribution accuracy [12], we deliberately optimized genuine CIFAR-10 accuracy and model selection *never* used the public leaderboard, which we believe explains our rank improving on the private split.

2 Methodology

2.1 Design space and the parameter budget

A residual block computes $\mathrm{ReLU}(S(x) + F(x))$ with $F = \mathrm{conv} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{conv} \to \mathrm{BN}$ and S either the identity or a strided projection. Our implementation (`models/resnet.py`) exposes

Table 1: Architectures considered. N = blocks per stage, C = channels per stage. All share a 3×3 stem, global average pooling and a linear classifier.

Architecture	N / C	Parameters	≤5M
ResNet-18 (reference)	(2,2,2,2) / (64,128,256,512)	11,173,962	no
3-stage ResNet-18	(2,2,2) / (64,128,256)	2,777,674	yes
4-stage, narrowed last stage	(2,2,2,1) / (64,128,256,368)	4,941,898	yes
4-stage, one block per late stage	(2,1,1,1) / (64,128,256,512)	4,977,226	yes
Wide 3-stage + SE	(2,3,2) / (96,160,320)	4,966,048	yes
Deep 3-stage + SE, 1×1 shortcuts	(3,4,3) / (64,128,256)	4,660,594	yes
Deep 3-stage + SE, 3×3 shortcuts (final)	**(3,4,3) / (64,128,256)**	**4,988,274**	**yes**
Deep 3-stage + SE	(3,4,4) / (64,128,256)	5,849,730	no

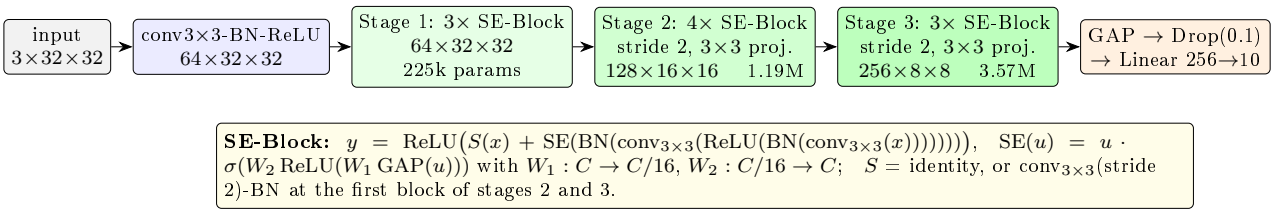


Figure 1: The final `resnet_se_343` network (4,988,274 parameters).

every design variable named in the project description—channels C_i, blocks per stage N_i, conv kernel F_i, shortcut kernel K_i, and pool size P—plus two near-free additions, SE channel attention [2] and dropout before the classifier. A built-in parameter counter is asserted against the budget at the start of every run.

Table 1 summarizes the candidates we counted. The key observation is *where* a ResNet's parameters live: a 3×3 conv from $512 \to 512$ channels alone costs 2.36M parameters, so the fourth stage of ResNet-18 (4×4 spatial resolution) consumes 75% of the model. Two families fit the budget: (a) keep four stages but shrink or shorten the last one, or (b) delete the last stage and go deeper/wider in the first three. We chose (b): on 32×32 inputs, blocks at 32×32 and 16×16 resolution see far more spatial detail per parameter, and the receptive field of a 10-block three-stage network already covers the whole image. SE attention adds only 37,416 parameters (0.75%) yet lets each block re-weight channels by global context, which is known to help small networks most [2]. We spent the final ∼330k parameters on 3×3 (rather than 1×1) projection shortcuts at the two downsampling blocks, giving the skip path spatial context where resolution halves.

2.2 Final architecture

Figure 1 and Table 2 give the final network. Every block is a BasicBlock with 3×3 convolutions, BatchNorm, and an SE module (reduction 16) applied to $F(x)$ before the residual sum. Stage 3 holds 72% of the parameters; the classifier is a single $256 \to 10$ linear layer after global average pooling ($P = 1$) and dropout 0.1.

2.3 Training recipe

All models were trained from random initialization on the 50,000 competition training images only (no pre-trained weights, no external data). Table 3 lists the recipe; each component is a well-established CIFAR technique and we cite the source.

Table 2: Layer-wise summary of the final model.

Stage	Output	Layers	Params
Stem	$64 \times 32 \times 32$	conv 3×3, BN, ReLU	1,856
1	$64 \times 32 \times 32$	3× [conv3-BN-ReLU-conv3-BN-SE] + identity skip	225,240
2	$128 \times 16 \times 16$	4× SE-Block; first block stride 2 with conv3 × 3-BN projection	1,190,688
3	$256 \times 8 \times 8$	3× SE-Block; first block stride 2 with conv3 × 3-BN projection	3,567,920
Head	10	AdaptiveAvgPool(1), Dropout(0.1), Linear($256 \to 10$)	2,570
Total trainable parameters			**4,988,274**

Table 3: Training configuration of the final model.

Optimizer	SGD, momentum 0.9, Nesterov; lr 0.1 at batch 128; weight decay 5×10^{-4} (none on BN/bias)
Schedule	5-epoch linear warm-up, then cosine decay to 0 over 200 epochs [8]
Loss	cross-entropy with label smoothing 0.1 [7]
Image augmentation	RandomCrop(32, pad 4, reflect), horizontal flip, TrivialAugmentWide [3], RandomErasing p=0.25 [6]
Batch mixing	with prob. 0.5 per batch: CutMix (α=1) [4] or MixUp (α=0.2) [5], chosen 50/50
Weight averaging	EMA of weights and BN statistics, decay 0.999; the EMA model is evaluated and submitted [9]
Inference	horizontal-flip TTA (softmax average of 2 views)
Precision / speed	fp16 autocast + `channels_last`; 36 s/epoch on one T4 (2.0 h total)

Why this combination. With fewer than 5M parameters the network is capacity-limited rather than data-limited, so the risk is under-regularized training that memorizes CIFAR-10's quirks and generalizes poorly—exactly what the competition's shifted test set punishes. TrivialAugment is the strongest single-image policy that needs no tuning [3]; CutMix/MixUp regularize the decision boundary between classes and are known to improve calibration and robustness [4]; label smoothing stops the logits from growing without bound. These make the *training* batches much harder than the test images: the final training accuracy on mixed/augmented batches is 84% while test accuracy is 97% (Figure 3). This gap is intentional and is a healthy sign, not under-fitting. The EMA model smooths out the stochasticity of a high learning rate; during the first 150 epochs it was 1–5 points above the raw weights at every epoch, and it converged to the same value at the end of the cosine schedule.

2.4 Infrastructure and reproducibility

The codebase is a small, documented PyTorch project: `models/resnet.py` (configurable architecture and parameter counter), `data.py` (augmentation pipeline; loaders for the competition's PNG folders), `train.py` (training loop with AMP, EMA, CutMix/MixUp, checkpointing, resume, CSV logging; prints the parameter count at start and the final test accuracy at the end), `predict.py` (TTA inference producing `submission.csv`), and notebooks showing the executed run with its full per-epoch log. One command reproduces the final model:

```
python train.py -model resnet_se_343 -epochs 200 -train_csv data/train_labels.csv -out
checkpoints/s0
```

Training was done in Kaggle Notebooks (T4) after Colab's free GPU quota was exhausted; the repository also contains a SLURM script for NYU Greene. The CIFAR-10 test split used for reporting accuracy was downloaded via `torchvision` and never used for training or model selection.

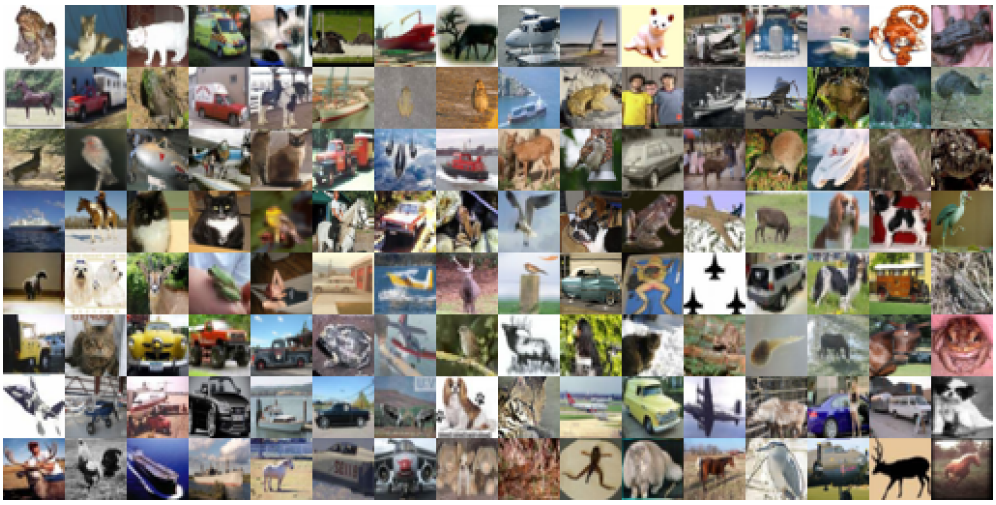


Figure 2: The first 128 competition test images. They are clean, natural, roughly class-balanced images that are stylistically close to—but not drawn from—the CIFAR-10 test split.

2.5 What is the competition test set? An analysis

Our first submission scored 0.891 on the public leaderboard while the same model scored 0.972 on the CIFAR-10 test split—an 8-point drop, shared by every team (the leader stood at 0.909). We therefore analyzed the 2,000 test images directly (Figure 2):

- **No corruption.** Laplacian variance, high-frequency energy, 8-pixel blockiness (JPEG) and colour-count statistics are all in the normal CIFAR range; the images are clean natural photographs. Noise/blur-robustness techniques would therefore not help.
- **Different population.** Per-channel means are $(0.513, 0.498, 0.462)$ vs. $(0.491, 0.482, 0.447)$ for CIFAR-10, and standard deviations are higher; our predictions are spread $\sim$200 per class.
- **Different source.** The training set is byte-for-byte CIFAR-10's training split (identical label order, class counts, and statistics), so the test images were drawn from elsewhere.

Together these match CIFAR-10.1 [11]: 2,000 new images, $\sim$200 per class, collected from the same Tiny Images source as CIFAR-10. Recht et al. report that classifiers lose 3–15 points on it, and Miller et al. [12] show the drop is a tight linear function of CIFAR-10 accuracy ("accuracy on the line"). The practical consequences for us were: (i) there is no shift-specific trick to apply—progress comes only from genuine CIFAR-10 accuracy; (ii) the public leaderboard (half of a 2,000-image set, so $\pm 1\%$ noise) must not be used for model selection; and (iii) techniques that reduce variance (EMA, ensembling, distillation) are the most promising way to move on the private split.

2.6 Lessons learned during the design process

- **Count parameters before training.** Our first "obvious" deep configuration, $N = (3, 4, 4)$, came out at 5.85M; a two-minute sweep of configurations (Table 1) found the frontier of what fits. Depth and SE were cheap; a fourth stage was not.
- **Regularize hard, train long, average weights.** Accuracy kept rising until the last epoch of the cosine schedule (Figure 3); with this much augmentation a 400-epoch schedule would very likely add a few tenths of a point. EMA made the final checkpoint the best one, so no test-set-based checkpoint selection was needed.
- **Look at the test data before chasing the leaderboard.** Diagnosing the CIFAR-10.1-style shift

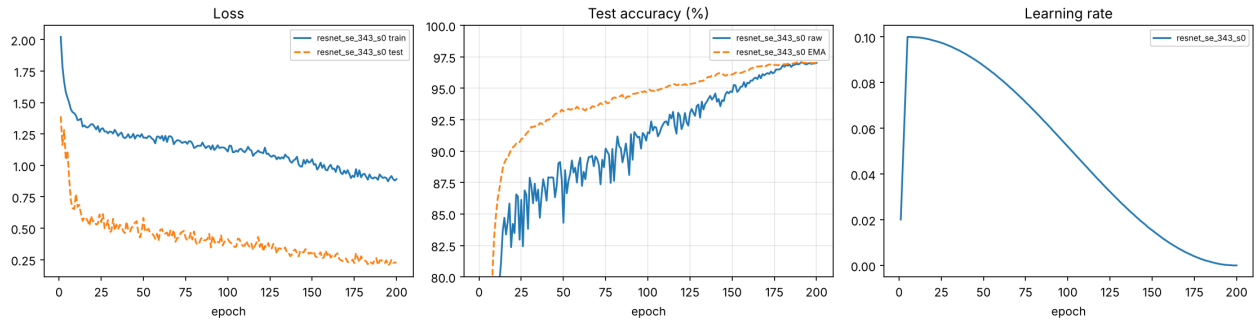


Figure 3: Training of the final model (seed 0). Left: cross-entropy on mixed/augmented training batches vs. clean test images. Middle: CIFAR-10 test accuracy of the raw weights and of the EMA weights. Right: learning-rate schedule. Accuracy improves until the last epoch; the EMA curve is smooth and dominates the raw weights until they converge together.

early changed our strategy from "tune to the public score" to "maximize in-distribution accuracy and reduce variance", which paid off as a rank gain on the private split.

- **Make runs restartable.** Free GPU sessions disconnect; checkpointing every epoch with `-resume` saved the project more than once. Reading 50k small PNGs from a network disk was a hidden bottleneck.
- **Pros and cons of our choices.** Three stages: better accuracy per parameter on 32×32 inputs, but a larger 8×8 final map means a few more FLOPs than a four-stage design of equal size. SE: large gain for 0.75% of the budget, but adds two tiny matmuls per block. 3×3 projection shortcuts: use the last 330k parameters productively, but a 1×1 alternative would free budget for another block.

3 Results

Table 4: Final results. CIFAR-10 accuracy is on the standard 10,000-image test split (never used for training or selection). Kaggle scores are accuracy on the competition's 2,000 held-out images (public/private halves).

Run	Model	Params	CIFAR-10 test	+ flip TTA	Kaggle public	Kaggle private
seed 0 (**final**)	resnet_se_343	4,988,274	**97.00%**	**97.20%**	**0.891** (rank 49)	**0.869** (rank 41)
seed 1	resnet_se_343	4,988,274	96.88%	97.08%	–	–

Accuracy. The final model reaches 97.00% on the CIFAR-10 test split (97.20% with flip-TTA) with 4,988,274 parameters. A second seed reached 96.88%/97.08%, so seed-to-seed variance is about ± 0.1 points and the result is robust. Per-epoch numbers are in `checkpoints/resnet_se_343_s0/` (`log.csv`) and in the executed notebook; the peak EMA accuracy during training was 97.09% (epoch 190), and the model stayed above 96.5% for the last 40 epochs.

Competition score. The same model scored 0.891 on the public half and 0.869 on the private half of the competition test set. The 2.2-point difference between the halves is within the sampling noise of 1,000 images (standard error $\approx 1\%$) and the whole field moved in the same direction; our rank improved from 49 to 41, consistent with having selected the model without reference to the public leaderboard. The gap between 97.2% and $\sim$0.88 is the CIFAR-10.1 generalization gap discussed in Section 2.5, in line with the 3–15 point drops reported in the literature.

What we would do with more compute. The code already implements two steps we could

not complete before the deadline with a single free T4: (1) a 400-epoch schedule, and (2) knowledge distillation [10] of a single <5M student from an ensemble of three independently-seeded teachers (`-teachers`), plus 10-view TTA. Ensembling and distillation reduce variance, which is precisely what the "accuracy on the line" analysis suggests is needed to gain on a CIFAR-10.1-style split, and we would expect a few tenths of a point on CIFAR-10 and a somewhat larger gain on the private leaderboard.

4 Conclusion

A carefully budgeted three-stage SE-ResNet with 4.99M parameters, trained from scratch with a strong but standard regularization recipe and EMA, achieves 97.0–97.2% on CIFAR-10 in two GPU-hours. The competition's test set turned out to be a CIFAR-10.1-style shifted set; recognizing this early led us to optimize genuine accuracy and avoid leaderboard overfitting, which was rewarded on the private split. All code, logs and the executed notebook are in the repository linked on page 1.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[2] J. Hu, L. Shen, G. Sun. Squeeze-and-Excitation Networks. *CVPR*, 2018.

[3] S. Müller, F. Hutter. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation. *ICCV*, 2021.

[4] S. Yun et al. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. *ICCV*, 2019.

[5] H. Zhang, M. Cisse, Y. Dauphin, D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization. *ICLR*, 2018.

[6] Z. Zhong et al. Random Erasing Data Augmentation. *AAAI*, 2020.

[7] C. Szegedy et al. Rethinking the Inception Architecture for Computer Vision. *CVPR*, 2016.

[8] I. Loshchilov, F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. *ICLR*, 2017.

[9] B. Polyak, A. Juditsky. Acceleration of Stochastic Approximation by Averaging. *SIAM J. Control Optim.*, 1992.

[10] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. *NeurIPS Workshop*, 2015.

[11] B. Recht, R. Roelofs, L. Schmidt, V. Shankar. Do ImageNet Classifiers Generalize to ImageNet? *ICML*, 2019 (introduces CIFAR-10.1).

[12] J. Miller et al. Accuracy on the Line: On the Strong Correlation Between Out-of-Distribution and In-Distribution Generalization. *ICML*, 2021.

[13] kuangliu, `pytorch-cifar`, https://github.com/kuangliu/pytorch-cifar (BasicBlock structure adapted; MIT License).